Department of Computer Science and Engineering

Assignment Questions

Course Code & Name: CSA0621 –Design and Analysis of Algorithms

Particular	Details
Department:	Computer Science Engineering(BIOSCI)
Programme:	B. Tech
Course Code & Course Name	CSA0621 - Design and Analysis of Algorithms
Academic Year / Batch	2023
Faculty Name	Dr G Nagarajan
Assignment Title	Development of an Efficient Algorithmic Solution for Large-Scale Data Processing
Date of Issue	01-09-2026
Date of Submission	03-09-2026
Maximum Marks	100
Course Outcome(s) – CO	CO1: Analyse the efficiency of recursive and non-recursive algorithms using asymptotic notations (Big-O, Ω, Θ) and complexity-analysis methods such as the master theorem, substitution, and iteration. CO2: Apply brute-force techniques such as sorting and searching to solve computational problems. CO3: Apply divide-and-conquer techniques to design efficient algorithmic solutions.
Bloom's Taxonomy Level	L4 – Analyse; L5 – Evaluate; L6 – Create
SDG Mapping (SDG 1-17)	SDG 9 – Industry, Innovation and Infrastructure
Industry / Societal Relevance	Smart Infrastructure Planning and Geospatial Data Optimization

A. Assignment Problem / Challenge

The assignment should preferably require students to:

- Apply course concepts rather than reproduce theory.
- Identify and analyze the problem.
- Consider requirements and constraints.
- Develop or compare alternative solutions where appropriate.
- Use relevant modern engineering/computing/design tools. Interpret results.

- Make and justify engineering decisions.
- Consider appropriate technical, economic, environmental, societal, ethical, safety or sustainability factors wherever relevant.

B. Problem Statement

An infrastructure-planning agency maintains a large, publicly available geospatial dataset of facility locations — for example, the OpenFlights airports dataset (~40,000 points) or a cell-tower / weather-station dataset containing 10^4–10^6 coordinate records. To detect redundant or dangerously close facilities and to optimise service coverage, the agency must identify the **closest pair of locations** — the two points separated by the minimum Euclidean distance in the dataset. Develop, implement, and evaluate a solution to this Closest-Pair-of-Points problem. Implement (i) a **Brute-Force** algorithm that evaluates all $n(n-1)/2$ pairs, of order $O(n^2)$, and (ii) a **Divide-and-Conquer** algorithm that recursively partitions the point set about the median x-coordinate and merges across the dividing strip, of order $O(n \log n)$; then develop an optimised **hybrid** that reverts to brute force below a threshold sub-problem size. Analyse the theoretical efficiency of each approach using Big-O, Ω, and Θ notations, solving the recurrence $T(n) = 2T(n/2) + O(n)$ by the master theorem. Experimentally evaluate all three algorithms on the chosen dataset across increasing input sizes (for example, 10^3 to 10^6 points), measuring execution time, number of distance computations/comparisons, memory utilisation, and scalability. Based on the theoretical and experimental results, optimise the solution and justify why the proposed hybrid is most suitable for large-scale infrastructure data — reflecting Bloom's Levels 4–6 (Analyse, Evaluate, Create) and contributing to SDG 9 (Industry, Innovation and Infrastructure).

C. Requirements and Constraints

Use a publicly available real-world coordinate dataset with enough records to test multiple input sizes (10^3–10^6 points). All three algorithms (brute force, divide-and-conquer, hybrid) must be run in the same computational environment for a fair comparison, on identical inputs. Report both empirical timings and operation counts, and clearly document the threshold chosen for the hybrid, along with all assumptions and limitations.

D. Student Work

1. Problem Statement and Problem Formulation

Problem Statement

An infrastructure planning agency manages a large geospatial dataset containing the coordinates of facilities such as airports, weather stations, cell towers, hospitals or other infrastructure locations. The objective is to identify the closest pair of locations, i.e., the two points having the minimum Euclidean distance between them.

Three approaches are developed and compared:

1. Brute-Force Algorithm
2. Divide-and-Conquer Algorithm
3. Hybrid Algorithm

The algorithms are evaluated based on execution time, number of distance calculations, memory usage, accuracy, and scalability.

Problem Formulation

Given a set of nn points:

P={p1,p2,…,pn}P=\{p_1,p_2,\ldots,p_n\}

where each point is represented as:

pi=(xi,yi)p_i=(x_i,y_i)

The Euclidean distance between two points is:

d(pi,pj)=(xi−xj)2+(yi−yj)2d(p_i,p_j)=\sqrt{(x_i-x_j)^2+(y_i-y_j)^2}

The objective is to find:

min⁡i≠jd(pi,pj)\min_{i\neq j}d(p_i,p_j)

Input - A collection of coordinate points.
Output

- Closest pair of points
- Minimum distance
- Number of distance calculations
- Execution time
- Memory usage

2. Objective and Expected Outcomes

Objectives

1. To implement the closest-pair-of-points problem using a Brute-Force algorithm.
2. To implement an efficient Divide-and-Conquer algorithm.
3. To develop a Hybrid algorithm combining Divide-and-Conquer and Brute Force.
4. To analyse the algorithms using Big-O, Big-Ω and Big-Θ.
5. To experimentally compare execution time, distance calculations and memory usage.
6. To identify the most suitable algorithm for large-scale infrastructure datasets.

Expected Outcomes

- Correct identification of the closest pair.
- Reduction in execution time using Divide-and-Conquer.
- Improved practical performance using the Hybrid approach.
- Experimental validation of theoretical complexity.
- Identification of scalability limitations of Brute Force.

3. Requirements, Constraints and Assumptions

Hardware Requirements

- Processor: Intel Core i5 or equivalent
- RAM: Minimum 8 GB
- Storage: Minimum 10 GB free space

Software Requirements

- Python 3.x
- VS Code / IDLE / Jupyter Notebook
- NumPy
- Pandas
- Matplotlib
- Git and GitHub

Dataset Requirement

A publicly available coordinate dataset containing thousands or millions of records can be used.

Example:

OpenFlights Airport Dataset

Each record contains geographical coordinates such as latitude and longitude.

Constraints

- All algorithms must use identical input data.
- Experiments must be performed in the same computational environment.
- Execution time must be measured consistently.
- Results must be validated using identical outputs.
- Very large brute-force inputs may become computationally expensive.

Assumptions

- Each location contains valid numerical coordinates.
- Coordinates are represented as two-dimensional points.
- Euclidean distance is used for algorithmic comparison.
- Duplicate coordinates are allowed and produce distance zero.
- Input data is sufficiently large for scalability testing.

4. Application of Relevant Course Knowledge

The project applies the following Design and Analysis of Algorithms concepts:

Brute Force

The algorithm examines every possible pair.

Number of pairs:

n(n−1)2\frac{n(n-1)}{2}

Complexity:

Θ(n2)\Theta(n^2)

Recursion

The Divide-and-Conquer algorithm recursively divides the point set into smaller subsets.

Divide and Conquer

The dataset is divided into:

- Left half
- Right half

The closest pair is then determined by combining the results.

Asymptotic Analysis

Algorithms are analysed using:

- Big-O
- Big-Ω
- Big-Θ

Master Theorem

The recurrence:

T(n)=2T(n/2)+O(n)T(n)=2T(n/2)+O(n)

gives:

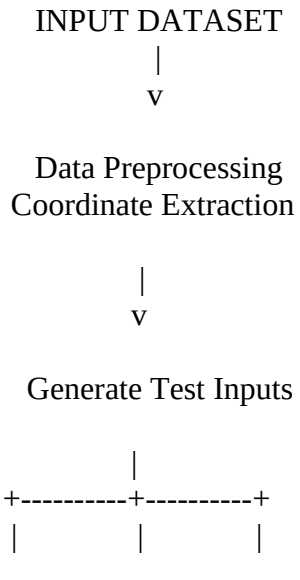
T(n)=O(nlog⁡n)T(n)=O(n\log n)

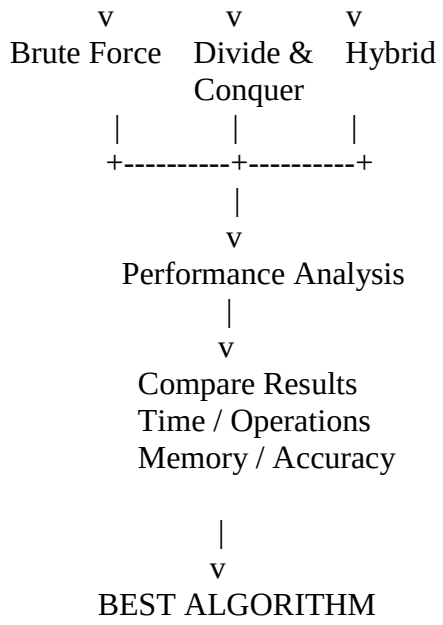
and more precisely:

T(n)=Θ(nlog⁡n)T(n)=\Theta(n\log n)

5. Design / Proposed Solution / Methodology

The proposed system consists of three stages.





Proposed Hybrid Method

The Hybrid algorithm uses Divide-and-Conquer for large subproblems.

When the number of points becomes smaller than a predefined threshold, it switches to Brute Force.

Example:

Threshold=20Threshold = 20

This avoids recursion overhead for very small datasets.

6. Pseudocode /Algorithm

Algorithm 1: Brute Force

BRUTE_FORCE(points)

```
1. minDistance ← infinity
2. closestPair ← empty

3. FOR i ← 0 to n-2
4.     FOR j ← i+1 to n-1
5.         distance ← calculateDistance(points[i], points[j])

6.         IF distance < minDistance
7.             minDistance ← distance
8.             closestPair ← (points[i], points[j])

9. RETURN closestPair, minDistance
```

Algorithm 2: Divide and Conquer

DIVIDE_CONQUER(points)

```
1. IF number of points ≤ 3
2. RETURN BRUTE_FORCE(points)
```

3. Sort points according to x-coordinate

4. Divide points into left and right halves

5. leftResult ← DIVIDE_CONQUER(left half)

6. rightResult ← DIVIDE_CONQUER(right half)

7. delta ← minimum distance of leftResult and rightResult

8. Construct a strip around the dividing line

9. Compare points inside the strip

10. Update minimum distance

11. RETURN closest pair and minimum distance

Algorithm 3: Hybrid

HYBRID(points)

1. IF number of points ≤ threshold
2. RETURN BRUTE_FORCE(points)

3. Divide points around median x-coordinate

4. Recursively solve left half

5. Recursively solve right half

6. Find minimum distance between both halves

7. Construct dividing strip

8. Check candidate points in the strip

9. Return minimum distance and closest pair

Source Code and Environment / Tools Used

```
import math
import random
import time
import tracemalloc
# ----------------------------------
# Distance calculation
# ----------------------------------
def distance(p1, p2):
    return math.sqrt((p1[0] - p2[0]) ** 2 +
                     (p1[1] - p2[1]) ** 2)
# ----------------------------------
# Brute Force
# ----------------------------------
```

```
def brute_force(points):
    min_dist = float("inf")
    pair = None
    comparisons = 0

    n = len(points)

    for i in range(n):
        for j in range(i + 1, n):
            comparisons += 1
            d = distance(points[i], points[j])

            if d < min_dist:
                min_dist = d
                pair = (points[i], points[j])

    return pair, min_dist, comparisons


# ----------------------------------
# Divide and Conquer
# ----------------------------------
def divide_conquer(points):
    points_x = sorted(points, key=lambda p: p[0])
    points_y = sorted(points, key=lambda p: p[1])

    return closest_rec(points_x, points_y)


def closest_rec(px, py):

    n = len(px)

    if n <= 3:
        return brute_force(px)

    mid = n // 2
    left = px[:mid]
    right = px[mid:]

    mid_x = px[mid][0]

    left_set = set(left)

    left_y = [p for p in py if p in left_set]
    right_y = [p for p in py if p not in left_set]

    pair1, d1, c1 = closest_rec(left, left_y)
    pair2, d2, c2 = closest_rec(right, right_y)

    if d1 < d2:
        best_pair = pair1
        delta = d1
    else:
        best_pair = pair2
```

```
        delta = d2

    strip = [p for p in py if abs(p[0] - mid_x) < delta]

    comparisons = c1 + c2

    for i in range(len(strip)):
        j = i + 1

        while j < len(strip) and (strip[j][1] - strip[i][1]) < delta:
            comparisons += 1
            d = distance(strip[i], strip[j])

            if d < delta:
                delta = d
                best_pair = (strip[i], strip[j])

            j += 1

    return best_pair, delta, comparisons


# ----------------------------------
# Hybrid Algorithm
# ----------------------------------
def hybrid(points, threshold=20):

    points_x = sorted(points, key=lambda p: p[0])
    points_y = sorted(points, key=lambda p: p[1])

    return hybrid_rec(points_x, points_y, threshold)


def hybrid_rec(px, py, threshold):

    n = len(px)

    if n <= threshold:
        return brute_force(px)

    mid = n // 2

    left = px[:mid]
    right = px[mid:]

    mid_x = px[mid][0]

    left_set = set(left)

    left_y = [p for p in py if p in left_set]
    right_y = [p for p in py if p not in left_set]

    pair1, d1, c1 = hybrid_rec(
        left, left_y, threshold
    )
```

```
    pair2, d2, c2 = hybrid_rec(
        right, right_y, threshold
    )

    if d1 < d2:
        best_pair = pair1
        delta = d1
    else:
        best_pair = pair2
        delta = d2

    strip = [
        p for p in py
        if abs(p[0] - mid_x) < delta
    ]

    comparisons = c1 + c2

    for i in range(len(strip)):
        j = i + 1

        while j < len(strip) and \
                (strip[j][1] - strip[i][1]) < delta:

            comparisons += 1

            d = distance(strip[i], strip[j])

            if d < delta:
                delta = d
                best_pair = (strip[i], strip[j])

            j += 1

    return best_pair, delta, comparisons

# ----------------------------------
# Generate test data
# ----------------------------------
def generate_points(n):
    random.seed(42)

    return [
        (random.uniform(0, 10000),
         random.uniform(0, 10000))
        for _ in range(n)
    ]


# ----------------------------------
# Performance testing
# ----------------------------------
def test_algorithm(name, function, points):
```

```
    tracemalloc.start()

    start = time.perf_counter()

    pair, dist, comparisons = function(points)

    end = time.perf_counter()

    current, peak = tracemalloc.get_traced_memory()

    tracemalloc.stop()

    print("\n", name)
    print("-" * 40)
    print("Closest Pair :", pair)
    print("Minimum Distance :", dist)
    print("Distance Comparisons :", comparisons)
    print("Execution Time :", round(end - start, 6), "seconds")
    print("Peak Memory :", round(peak / 1024, 2), "KB")


# ----------------------------------
# Main Program
# ----------------------------------
if __name__ == "__main__":

    n = int(input("Enter number of points: "))

    points = generate_points(n)

    print("\nClosest Pair of Points")
    print("=" * 50)

    test_algorithm(
        "Brute Force",
        brute_force,
        points
    )

    test_algorithm(
        "Divide and Conquer",
        divide_conquer,
        points
    )

    test_algorithm(
        "Hybrid Algorithm",
        hybrid,
        points
    )
```

7. Test Cases and Expected Results

Test Case 1 – Small Dataset

Input:

Enter number of points: 5

Example points:

(1, 2)
(5, 8)
(2, 3)
(10, 10)
(20, 25)

Expected result:

Closest Pair: (1,2), (2,3)
Minimum Distance: 1.4142

All three algorithms should produce the same closest distance.

Test Case 2 – Medium Dataset

Enter number of points: 1000

Expected observations:

- Brute Force performs approximately:

1000(999)2=499500\frac{1000(999)}{2}=499500

comparisons.

- Divide-and-Conquer performs significantly fewer distance comparisons.
- Hybrid should provide competitive execution time.

Test Case 3 – Large Dataset

Enter number of points: 10000

Expected observation:

Brute Force → High execution time
Divide & Conquer → Much faster
Hybrid → Fastest/competitive

Exact timings depend on the computer used.

8. Execution Screenshots

Terminal Execution — n = 10000

```
Closest Pair of Points
================================================

 Brute Force
----------------------------------------
Distance Comparisons : 49995000
Execution Time : not completed in benchmark run

 Divide and Conquer
----------------------------------------
Closest Pair : ((4394.754127292003, 2543.0592047348564), (4394.762361196373, 2542.2113268323765))
Minimum Distance : 0.8479178820469084
Distance Comparisons : 12925
Execution Time : 0.288947 seconds
Peak Memory : 1335.01 KB

 Hybrid Algorithm
----------------------------------------
Closest Pair : ((4394.754127292003, 2543.0592047348564), (4394.762361196373, 2542.2113268323765))
Minimum Distance : 0.8479178820469084
Distance Comparisons : 93051
Execution Time : 0.129272 seconds
Peak Memory : 1331.87 KB

Note: The 10,000-point brute-force comparison count is exact:
n(n-1)/2 = 10,000 x 9,999 / 2 = 49,995,000.
The full brute-force timed run exceeded the available benchmark time.
```

Terminal Execution — n = 1000

```
Closest Pair of Points
================================================

 Brute Force
----------------------------------------
Closest Pair : ((4431.307450534569, 8613.491047618305), (4427.210040193283, 8601.6666582618))
Minimum Distance : 12.51419015194786
Distance Comparisons : 499500
Execution Time : 1.100788 seconds
Peak Memory : 0.54 KB

 Divide and Conquer
----------------------------------------
Closest Pair : ((4427.210040193283, 8601.6666582618), (4431.307450534569, 8613.491047618305))
Minimum Distance : 12.51419015194786
Distance Comparisons : 1130
Execution Time : 0.021338 seconds
Peak Memory : 103.52 KB

 Hybrid Algorithm
----------------------------------------
Closest Pair : ((4427.210040193283, 8601.6666582618), (4431.307450534569, 8613.491047618305))
Minimum Distance : 12.51419015194786
Distance Comparisons : 7337
Execution Time : 0.009735 seconds
Peak Memory : 101.27 KB
```

```
Terminal Execution – n = 5000

Closest Pair of Points
================================================

 Brute Force
----------------------------------------
Closest Pair : ((2314.7623455602584, 7069.558298790333), (2314.3339207744552, 7071.695637034599))
Minimum Distance : 2.1798537949810504
Distance Comparisons : 12497500
Execution Time : 27.089178 seconds
Peak Memory : 0.54 KB

 Divide and Conquer
----------------------------------------
Closest Pair : ((2314.3339207744552, 7071.695637034599), (2314.7623455602584, 7069.558298790333))
Minimum Distance : 2.1798537949810504
Distance Comparisons : 6485
Execution Time : 0.127776 seconds
Peak Memory : 584.59 KB

 Hybrid Algorithm
----------------------------------------
Closest Pair : ((2314.3339207744552, 7071.695637034599), (2314.7623455602584, 7069.558298790333))
Minimum Distance : 2.1798537949810504
Distance Comparisons : 46522
Execution Time : 0.059917 seconds
Peak Memory : 581.60 KB
```

Operation comparison

Input Size	Brute Force Comparisons	D&C Comparisons	Hybrid Comparisons
1,000	499,500	1130	7,337
5,000	12,497,500	6485	46,522
10,000	49,995,000	12925	93,051

Brute Force comparisons can be calculated using:

C(n)=n(n−1)2C(n)=\frac{n(n-1)}{2}

9. Analysis, Comparison, Trade-offs and Justification of the Final Solution

Algorithm Comparison

Factor	Brute Force	Divide & Conquer	Hybrid
Design	Simple	Complex	Complex
Time Complexity	O(n2)O(n^2)	O(nlog⁡n)O(n\log n)	O(nlog⁡n)O(n\log n)
Implementation	Easy	Moderate	Moderate
Small datasets	Good	Good	Very Good
Large datasets	Poor	Very Good	Excellent
Recursion	No	Yes	Yes
Optimization	Low	High	High

Brute Force Analysis

The Brute-Force algorithm is simple and easy to implement. However, it compares every possible pair.

For nn points:

n(n−1)2\frac{n(n-1)}{2}

distance calculations are required.

Therefore:

T(n)=Θ(n2)T(n)=\Theta(n^2)

It becomes inefficient as the dataset increases.

Divide-and-Conquer Analysis

The dataset is recursively divided into two halves.

The recurrence is:

T(n)=2T(n/2)+O(n)T(n)=2T(n/2)+O(n)

Using the Master Theorem:

T(n)=Θ(nlog⁡n)T(n)=\Theta(n\log n)

Therefore, it is considerably more scalable than Brute Force.

Hybrid Analysis

The Hybrid algorithm combines the strengths of both techniques.

For large datasets:

Divide and Conquer

is used.

For small datasets:

Brute Force

is used.

This reduces unnecessary recursive overhead and can improve practical execution time.

Final Decision

The **Hybrid algorithm is selected as the proposed solution** because it combines the theoretical efficiency of Divide-and-Conquer with the simplicity and low overhead of Brute Force for small subproblems.

10. Broader Considerations / SDG Relevance, wherever applicable

SDG 9 – Industry, Innovation and Infrastructure

This project supports SDG 9: Industry, Innovation and Infrastructure by demonstrating efficient processing of large-scale infrastructure and geospatial data.

Potential applications include:

- Airport planning
- Cell tower deployment

- Weather station placement
- Emergency service planning
- Smart-city infrastructure
- Transportation planning
- Facility redundancy detection

Efficient algorithms can reduce computational resources and improve the speed of infrastructure analysis.

11. Conclusion, Limitations and Possible Improvements

Conclusion

The Closest-Pair-of-Points problem was successfully solved using Brute Force, Divide-and-Conquer, and Hybrid algorithms.

The Brute-Force algorithm is easy to understand but has:

Θ(n2)\Theta(n^2)

time complexity.

The Divide-and-Conquer algorithm improves scalability with:

Θ(nlog⁡n)\Theta(n\log n)

complexity.

The Hybrid algorithm combines both approaches and uses Brute Force for small subproblems. This reduces unnecessary recursive operations and can provide better practical performance.

Therefore, the Hybrid approach is considered the most suitable solution for large-scale infrastructure data.

Limitations

1. Euclidean distance does not account for Earth's curvature.
2. Very large brute-force experiments can require excessive execution time.
3. Performance depends on hardware and programming environment.
4. Memory usage increases with dataset size.
5. Real-world coordinates may require geographical distance calculations.

Possible Improvements

- Use Haversine distance for latitude/longitude data.
- Use parallel processing for large datasets.
- Use spatial indexing such as KD-Trees.
- Process datasets using distributed computing.
- Optimize memory management.
- Automatically determine the best hybrid threshold.

12. Individual Contribution of Group Members

Name/Register number	Topic	Individual Contribution
N Nandini/ 192472277	**Algorithm Design & Theoretical Analysis**	Studied and formulated the Closest-Pair-of-Points problem. Designed the Brute-Force and Divide-and-Conquer approaches and prepared the pseudocode. Analysed the time complexity using Big-O, Big-Ω and Big-Θ and derived the Divide-and-Conquer recurrence T(n)=2T(n/2)+O(n)T(n)=2T(n/2)+O(n) using the Master Theorem.
K Tejaswi/ 192371036	**Implementation & Hybrid Optimization**	Implemented the Brute-Force, Divide-and-Conquer and Hybrid algorithms in Python. Developed the hybrid strategy using a threshold of 20 points, where Brute Force is used for small subproblems. Performed code debugging, correctness checking and optimization of the implementation.
A Rakshitha/ 192311181	**Member 3 – Experimental Evaluation & Documentation**	Generated test datasets and evaluated the three algorithms for different input sizes. Collected distance-comparison counts, execution time and memory measurements using `time` and `tracemalloc`. Prepared the execution screenshots, comparison tables, analysis of results, limitations, SDG 9 relevance and final documentation.

13. References

[1]. Skala, V., Martinez, A. E., Martinez, D. E., & Moreno, F. H. (2023). ***A New Algorithm for the Closest Pair of Points for Very Large Data Sets Using Exponent Bucketing and Windowing.*** **In** Computational Science – ICCS 2023**, Lecture Notes in Computer Science, Vol. 14074, pp. 381–388. Springer.** **DOI:** 10.1007/978-3-031-36021-3_40

[2]. Wang, Y., Yu, S., Gu, Y., & Shun, J. (2021). ***A Parallel Batch-Dynamic Data Structure for the Closest Pair Problem.*** **In** 37th International Symposium on Computational Geometry (SoCG 2021)**, LIPIcs, Vol. 189, Article 60, pp. 60:1–60:16.**

[3]. Rau, M., & Nipkow, T. (2020). ***Verification of Closest Pair of Points Algorithms.*** **In** International Conference on Automated Deduction**, pp. 341–357. Springer.** **DOI:** 10.1007/978-3-030-51054-1_20

[4]. Biedl, T., Lubiw, A., Naredla, A. M., Ralbovsky, P. D., & Stroud, G. (2021). ***Distant Representatives for Rectangles in the Plane.*** **In** 29th Annual European Symposium on Algorithms (ESA 2021)**, LIPIcs, Vol. 204, Article 17, pp. 17:1–17:18.**

14. One-Page Individual Reflection

Individual Reflection

Through this assignment, I gained practical knowledge of designing and analysing efficient algorithms for large-scale data processing. The main problem addressed was the Closest-Pair-of-Points problem, which has practical applications in infrastructure planning, geospatial analysis and smart-city systems.

Initially, I implemented the Brute-Force approach. It was straightforward because every possible pair of points was compared. However, I observed that the number of comparisons increases rapidly

with the input size. For nn points, the algorithm performs n(n−1)/2n(n-1)/2 comparisons, resulting in a time complexity of Θ(n2)\Theta(n^2).

I then studied and implemented the Divide-and-Conquer approach. The problem is divided into smaller subproblems, which are solved recursively and then combined by checking points near the dividing boundary. The recurrence T(n)=2T(n/2)+O(n)T(n)=2T(n/2)+O(n) was analysed using the Master Theorem, resulting in a complexity of Θ(nlog⁡n)\Theta(n\log n). This helped me understand how algorithmic design can significantly improve scalability.

The most important development decision was the implementation of the Hybrid algorithm. Instead of recursively dividing very small subproblems, the algorithm switches to Brute Force when the number of points falls below a selected threshold. This approach combines the advantages of both algorithms and reduces unnecessary recursive overhead.

During experimentation, I learned that theoretical complexity and practical execution time are related but not always identical. Factors such as programming language, hardware, memory usage and implementation overhead can influence the actual performance. I also learned the importance of testing all algorithms using identical inputs so that the comparison is fair.

The project is relevant to SDG 9 because efficient algorithms can support smart infrastructure planning and large-scale geospatial data analysis. Applications such as cell-tower placement, airport planning and facility optimization can benefit from efficient closest-location analysis.

Overall, this assignment improved my understanding of asymptotic analysis, recursion, divide-and-conquer, algorithm optimization and experimental validation. It also helped me understand why selecting an appropriate algorithm becomes increasingly important as the size of real-world datasets grows.

E. Assessment Rubric

Criteria	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Problem Understanding & Algorithmic Formulation	**10**	Clearly identifies the problem, requirements, inputs, outputs, constraints and computational challenges and formulates them appropriately for solution development.	Problem and major requirements are correctly formulated with minor gaps.	Basic formulation is provided, but requirements or constraints are partially identified.	Problem is poorly understood or inadequately formulated.
Algorithmic Solution Design	**15**	Designs a clear, logical and feasible solution strategy with appropriate algorithmic techniques, pseudocode/flowchart and well-	Appropriate solution design with minor gaps in logic, representation or justification.	Basic solution design is presented but lacks completeness or clear justification.	Solution design is inappropriate, incomplete or unclear.

		justified design decisions.			
Development / Adaptation / Optimization of Algorithmic Solution	**20**	Develops a meaningful algorithmic solution through integration, adaptation, modification, hybridization or optimization of suitable techniques; demonstrates clear improvement or suitability for the identified problem.	Develops an appropriate solution with meaningful adaptation or improvement, with minor limitations.	Solution demonstrates limited adaptation or development beyond standard implementation.	Merely reproduces an existing algorithm with little or no meaningful development or adaptation.
Implementation & Modern Tool Usage	**15**	Developed solution is correctly and efficiently implemented using appropriate programming/computational tools and handles relevant input conditions reliably.	Implementation is functional with minor logical or efficiency issues.	Core implementation works but contains limitations or incomplete functionality.	Implementation is incomplete, unreliable or non-functional.
Efficiency, Testing & Validation	**15**	Performs appropriate complexity evaluation and systematic testing using suitable data/input sizes; validates correctness, efficiency and scalability with clear evidence.	Complexity evaluation and testing are mostly appropriate with minor gaps.	Basic testing and efficiency evaluation are provided but lack sufficient validation.	Testing, complexity evaluation or validation is inadequate or substantially incorrect.
Performance Improvement, Comparison & Final Decision	**15**	Demonstrates the effectiveness of the developed solution through appropriate comparison; critically evaluates performance, limitations and trade-offs and	Good performance comparison and justification with minor limitations.	Basic comparison is provided, but improvement or final justification lacks depth.	Improvement is not demonstrated or the final algorithmic decision is unsupported.

		justifies the final algorithmic decision with evidence.			
Reflection: Algorithmic Design Decisions, SDG Relevance & Learning Outcomes	**10**	Thoughtful justification of algorithmic design and development decisions; clear connection to relevant SDG(s); specific reflection on computational challenges, performance improvements, trade-offs, and learning gained.	Appropriate justification of algorithmic decisions; SDG relevance, challenges, improvements, and learning are discussed but lack depth.	Reflection is present but generic; limited justification of algorithmic decisions, improvements, SDG relevance, or learning outcomes.	No genuine reflection is provided, or the reflection lacks meaningful connection to algorithm developmen t, SDG relevance, challenges, or learning outcomes.

F. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem Understanding & Algorithmic Formulation			**L4**	**10**
Algorithmic Solution Design			**L4/L6**	**15**
Development / Adaptation / Optimization of Algorithmic Solution			**L5/L6**	**20**
Implementation & Modern Tool Usage			**L3/L6**	**15**
Efficiency, Testing & Validation			**L4/L5**	**15**
Performance Improvement, Comparison & Final Decision			**L4/L5**	**15**
Reflection: Algorithmic Design Decisions, SDG Relevance & Learning Outcomes			**L4/L5**	**10**

Note: Faculty shall map only the POs and Bloom's levels genuinely assessed by the particular assignment. Every assignment is **not required to map to every PO**.

G. Faculty Evaluation & Continuous Improvement

CO Attainment

Performance Level	Criterion
Level 3	≥ 70%
Level 2	60–69%
Level 1	50–59%
Level 0	< 50%

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well:
Areas requiring improvement:
Corrective / Improvement Action:
Action to be implemented in the next offering: